Solution: Decode Ways

Problem Statement

You have given a string that consists only of digits. This string can be decoded into a set of alphabets where '1' can be represented as 'A', '2' as 'B', ... , '26' as 'Z'. The task is to determine how many ways the given digit string can be decoded into alphabets.

Examples

1. - **Input**: "121"
   - **Expected Output**: 3
   - **Justification**: The string "121" can be decoded as "ABA", "AU", and "LA".
2. - **Input**: "27"
   - **Expected Output**: 1
   - **Justification**: The string "27" can only be decoded as "BG".
3. - **Input**: "110"
   - **Expected Output**: 1
   - **Justification**: The string "110" can only be decoded as "JA".

Solution

Our approach to solving this problem involves using dynamic programming to iteratively build the solution. Given a string of digits, we want to determine how many ways it can be decoded into alphabets. The key insight is that the number of ways to decode a string of length `i` is dependent on the number of ways to decode the previous two substrings of length `i-1` and `i-2`. We'll use two variables, `prev` and `current`, to store these values and update them as we loop through the string.

1. **Initialization**: Begin by checking if the string is valid for decoding (e.g., it should not start with a '0'). If the string is invalid, return 0. Next, initialize two variables, `prev` and `current`, both set to 1. `prev` will store the number of ways to decode the string of length `i-2`, and `current` will store the number of ways to decode the string of length `i-1`.

2. **Iterate Through the String**: Loop through the string from the second character to the end. For each character, compute the number of ways it can be decoded when combined with the previous character.

3. **Update Variables**: For each character, evaluate the following conditions:
   - If the current character and the previous character form a valid number between 10 and 26, they can be decoded together.
   - If the current character is not '0', it can be decoded individually. Use these conditions to update the `prev` and `current` variables accordingly.

4. **Return the Result**: Once the iteration completes, the `current` variable will hold the total number of ways the entire string can be decoded. Return this value.

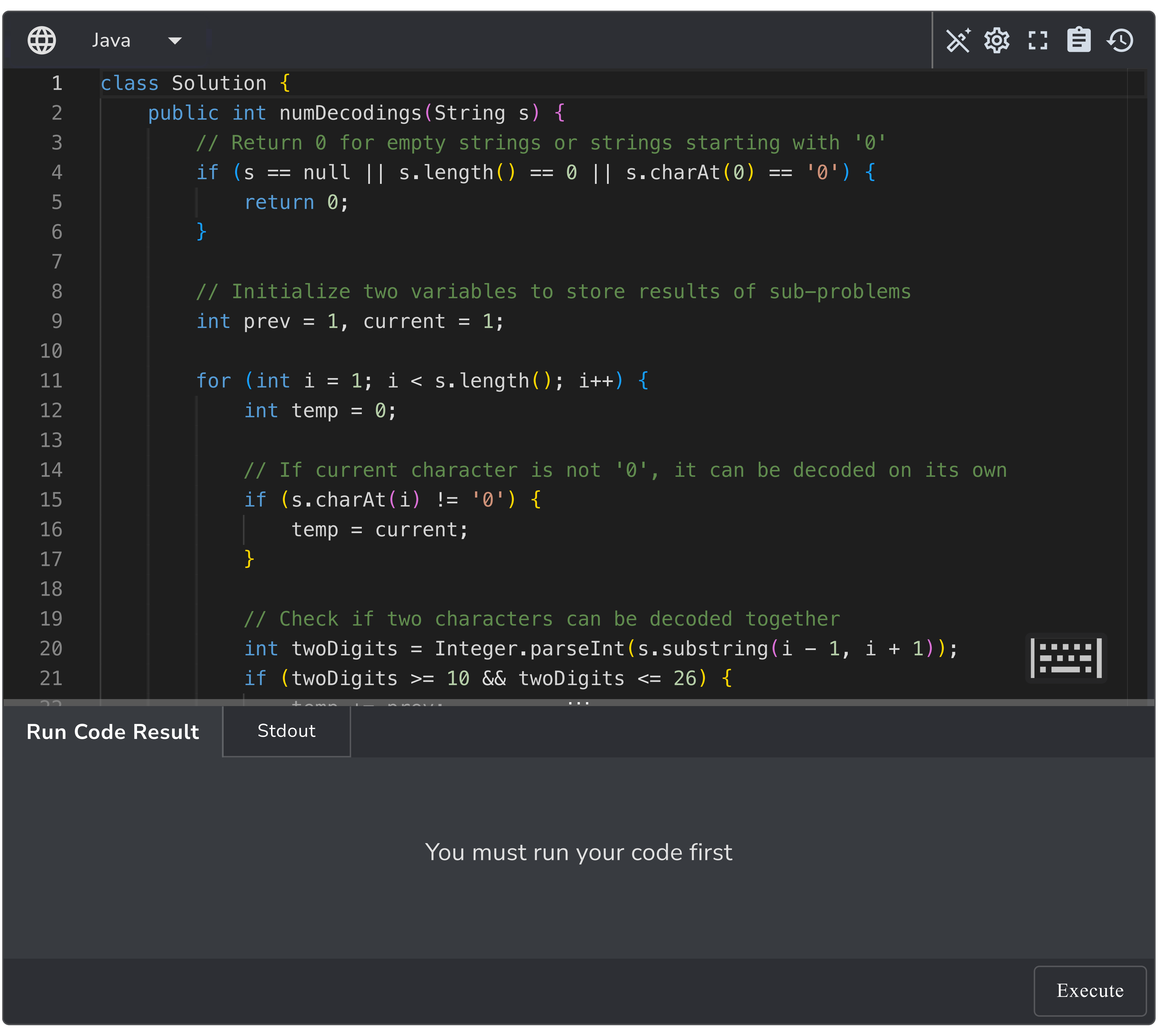
This dynamic programming approach is efficient because it computes the solution by using previously calculated results, and it avoids redundant calculations. By tracking and updating the number of ways to decode the current and previous substrings, the algorithm effectively builds the solution for the entire string.

Algorithm Walkthrough

Consider the input "121":

- Initialize `prev = 1` and `current = 1`.
- For the second character '2':
  - "12" can be decoded as "AB" or "L".
  - Update `prev` and `current` by swapping their values and setting `current += prev`.
  - Now, `prev = 1` and `current = 2`.
- For the third character '1':
  - "21" can be decoded as "BA" or "U".
  - Again, update `prev` and `current`.
  - Now, `prev = 2` and `current = 3`.
- The final answer is 3, which is the value of `current`.

Code



```java
class Solution {
    public int numDecodings(String s) {
        // Return 0 for empty strings or strings starting with '0'
        if (s == null || s.length() == 0 || s.charAt(0) == '0') {
            return 0;
        }

        // Initialize two variables to store results of sub-problems
        int prev = 1, current = 1;

        for (int i = 1; i < s.length(); i++) {
            int temp = 0;

            // If current character is not '0', it can be decoded on its own
            if (s.charAt(i) != '0') {
                temp = current;
            }

            // Check if two characters can be decoded together
            int twoDigits = Integer.parseInt(s.substring(i - 1, i + 1));
            if (twoDigits >= 10 && twoDigits <= 26) {
```

You must run your code first

Complexity Analysis

- **Time Complexity**: O(N), where N is the length of the string. We loop through the string once.
- **Space Complexity**: O(1), as we only use a constant amount of space regardless of the input size.

← Previous
Decode Ways (Medium)
Next →
Unique Paths (Medium)
☐ Mark as Completed